

QUESTION 9

Title:

Minimum Spanning Tree Using Kruskal's Algorithm

Aim

To find the Minimum Spanning Tree (MST) of a weighted graph using Kruskal's Algorithm and determine the minimum cost required to connect all nodes without forming cycles.

Problem Statement

A network engineer needs to connect multiple nodes in a network with the minimum possible cost. Each connection between nodes has an associated weight. The objective is to select a subset of edges that connects all nodes, avoids cycles, and minimizes the total cost.

Introduction

In computer networks, communication systems, transportation networks, and electrical grids, it is often necessary to connect multiple locations with minimum cost.

A Minimum Spanning Tree (MST) is a subset of graph edges that:

-
- Connects all vertices.
- Contains no cycles.
- Has the minimum possible total edge weight.

Kruskal's Algorithm is a greedy algorithm that constructs the MST by repeatedly selecting the smallest edge that does not form a cycle.

Objectives

- Understand the concept of Minimum Spanning Tree.
- Apply Kruskal's Algorithm to find the MST.
- Avoid cycle formation while connecting nodes.
- Minimize the total network cost.
- Analyze the algorithm's efficiency.

Theory

Minimum Spanning Tree (MST)

A spanning tree is a subset of a graph that connects all vertices without forming cycles.

A Minimum Spanning Tree is the spanning tree with the smallest possible total edge weight.

Properties of MST

- Connects all vertices.
- Contains exactly $(V - 1)$ edges.
- Contains no cycles.
- Has minimum total cost.

Kruskal's Algorithm

Kruskal's Algorithm follows the Greedy Method.

It always selects the edge with the smallest weight that does not create a cycle.

The algorithm continues until all vertices are connected.

Algorithm

Step 1

List all edges in the graph.

Step 2

Sort all edges in ascending order of weight.

Step 3

Select the smallest edge.

Step 4

Check whether adding the edge forms a cycle.

Step 5

If no cycle is formed, include the edge in the MST.

Step 6

Repeat until $(V - 1)$ edges are selected.

Pseudocode

KRUSKAL(G)

Sort all edges by increasing weight

FOR each edge (u,v)

IF adding edge does not form cycle

 Add edge to MST

RETURN MST

Input

Number of Vertices:

$n = 4$

Graph:

0 2 3 0

2 0 15 2

3 15 0 13

0 2 13 0

Graph Representation

Edge List:

$(0,1) = 2$

$(0,2) = 3$

$(1,2) = 15$

$(1,3) = 2$

$(2,3) = 13$

Working Procedure

Step 1: Sort Edges

Weight 2 $\rightarrow$ (0,1)

Weight 2 $\rightarrow$ (1,3)

Weight 3 $\rightarrow$ (0,2)

Weight 13 $\rightarrow$ (2,3)

Weight 15 $\rightarrow$ (1,2)

Step 2: Select Edges

Select (0,1) $\rightarrow$ Cost = 2

Select (1,3) $\rightarrow$ Cost = 2

Select (0,2) $\rightarrow$ Cost = 3

All vertices are connected.

Step 3: Calculate Total Cost

MST Cost = 2 + 2 + 3

MST Cost = 7

Source Code

```
graph = [  
    [0, 2, 3, 0],  
    [2, 0, 15, 2],  
    [3, 15, 0, 13],  
    [0, 2, 13, 0]  
]  
  
edges = [  
    (2, 0, 1),  
    (3, 0, 2),  
    (15, 1, 2),  
    (2, 1, 3),  
    (13, 2, 3)  
]  
  
parent = [0, 1, 2, 3]  
  
def find(x):  
    while parent[x] != x:  
        x = parent[x]  
    return x  
  
cost = 0  
  
for w, u, v in sorted(edges):  
    if find(u) != find(v):  
        parent[find(u)] = find(v)  
        cost += w  
  
print("MST Cost =", cost)
```

Sample Output

MST Cost = 7

Analysis

Why Kruskal's Algorithm?

- Simple to implement.
- Produces optimal solution.
- Efficient for sparse graphs.
- Uses a greedy strategy.

Advantages

- Easy to understand.
- Guarantees minimum cost.
- Efficient for large networks.
- Prevents cycle formation.
- Widely used in networking applications.

Disadvantages

- Requires sorting of edges.
- Slightly slower for dense graphs.
- Additional data structures may be needed for cycle detection.

Time Complexity

Sorting Edges:

$O(E \log E)$

Cycle Detection:

$O(E)$

Overall Complexity:

$O(E \log E)$

Space Complexity

$O(V)$

Where:

V = Number of Vertices

E = Number of Edges

Applications

- Computer Network Design
- Telecommunication Networks
- Road Construction Planning
- Electrical Power Distribution
- Water Supply Systems
- Transportation Networks

Result

The Minimum Spanning Tree was successfully generated using Kruskal's Algorithm. The selected edges connected all vertices without forming cycles and produced the minimum possible cost. The total MST cost obtained for the given graph is 7. Therefore, Kruskal's Algorithm provides an efficient and optimal solution for network design problems.